

What good Claude prompting looks like for the exam

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 8 of 12 · English notes

The second half of 19 August was a prompting lecture: be **clear and specific**, prefer **instructions over rambling questions**, use **action verbs**, add **guidelines** for length and constraints, mark data with **XML-style tags**, and then **evaluate** every template. Vijay framed evaluation as an AI developer's job, with a five-step Claude-oriented loop and three graders: human, code, or LLM-as-judge.

Clear beats long

More words are not more precision. Vague solar-panel rambling produced vague answers. Rewrite: "Write three paragraphs about how solar panels work." Lead with the task. Verbs he liked: write, generate, produce, identify, create, plan. Prefer "Identify three countries that use geothermal energy; include generation stats for each" over a story about something sounding neat.

A short-story prompt without constraints is also weak. Same task plus guidelines — under 1000 words, a revealing action, at least one supporting character — is the exam-ready version. Constraints are not optional flavour; they are the spec.

Claude-flavoured structure

- XML tags (user-invented) to label documents, rules, and untrusted input.
- Prefill + stop sequences when the consumer needs a tight envelope (JSON, fences).
- Few-shot examples inside the prompt when the format is brittle.
- System parameter for standing role / anti-hallucination (module 07), user content for the instance.
- Low temperature when you are extracting; higher when you are drafting prose.

The evaluation loop (five steps in class)

1. **Draft** a prompt (even a naive "Please answer the user's question: {q}").
2. **Build an eval set** of realistic questions — including on-distribution AWS tasks in the lab story, plus off-distribution junk ("what is 2+2", oatmeal) if you need to see refusal or tool-scope behaviour.
3. **Run** the set through the model; store prompt→response pairs.
4. **Grade** — not a boolean only: a score (e.g. out of 10) for completeness, format, faithfulness.
5. **Revise** the prompt (and maybe temperature / model) and repeat. Scores can go *down*; that is information.

The AWS internal-tool scenario: engineers describe a resource in English; Claude returns Python, JSON, or regex. Eval questions should include correct, wrong, and middling asks so you see how the draft behaves before you add "answer with ample detail" and other elaborations.

Three ways to grade

Grader	Good for	Cost / bias notes
Human / SME	Translation quality, brand voice	Slow; needed when you cannot specify a metric
Code (Pydantic, regex, JSON schema, tests)	Shape, types, “exactly one sentence”	Cheap, repeatable; does not know if facts are true
LLM-as-judge	Rubrics (faithfulness, reasoning quality) at scale	Use a capable model; prompt it <i>not</i> to substitute its own world knowledge; still biased; nested judges are not required

Judge models should behave like a “constitution”: apply your rubric, not rewrite the answer with their own French. Fine-tune the *judge prompt*; you do not train a new model for Foundation. Prefer Opus-class judges over a tiny generator if you must judge language — but Vijay’s cost warning: if everything is an LLM, the bill explodes. Implement code graders first. Anthropic’s platform has developer eval tooling; AWS Bedrock evaluations were shown as an analogue (Nova as judge, small model as generator, human eval as a paid service). Claude does not remove the need for your own loop.

Messages versus system — exam wording

Vijay said exams “mostly” ask how **messages** and the **system prompt** are configured on `create`. User and assistant live in the array; system is a sibling parameter. Do not invent a `role: system` inside messages unless the current docs say that form is accepted — class code used the top-level field.

Agent eval is bigger

The same draft→dataset→grade loop appears in agent testing, but agents add tools, RAG, and orchestration. Foundation written answers should keep prompt eval at developer scope unless the question names tools.

PLATFORM EVAL UIS CHANGE

Console “developer tests” URLs and Bedrock clicks from 19 August will rot. Concepts stay: dataset, generator model, judge rubric. Prefer [eval / guardrail docs](#) and prompting docs.

Pocket rubric for Foundation answers

- Task first, constraints next, data in tags.
- Show the format you want (or prefill it).
- Say how you would measure success with a grader.
- Name `stop_reason` / `max_tokens` if the output can truncate.
- Cite current Anthropic prompting docs, not a transcript anecdote, if a technique has a product name now (structured outputs, extended thinking).

Official documentation

- **Prompt engineering overview**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Clarity, XML tags, examples — the scoring key for this chapter.

- **Structured outputs**

<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

When the exam mentions guaranteed JSON.

- **Strengthen guardrails / eval**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Evaluation mindset beyond a single happy-path prompt.

- **Messages API**

<https://docs.claude.com/en/api/messages>

system versus messages configuration.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.